

Lectura 8: Amazon Aurora High Availability and Disaster Recovery Features for Global Resilience

Bases de datos II

Estudiante: Fabricio Monge Brenes Carnet: 2023108707

Instrucciones

1. ¿Cómo funciona la arquitectura de Spanner?
 2. ¿Cómo multi versionamiento en Spnnaer?
 3. ¿En qué consiste el TRUETIME?
-

Respuestas

¿Cómo funciona la arquitectura de Spanner?

Spanner es una base de datos distribuida globalmente, escalable, multiversión y replicada de forma síncrona. Su arquitectura se organiza de la siguiente manera:

- Universo: un despliegue de Spanner se llama universe. Google ejecuta pocos universos: uno de pruebas, uno de desarrollo/producción y uno solo de producción.
- Zonas: un universo se organiza en zonas. Una zona es la unidad de despliegue administrativo y de aislamiento físico; también es el conjunto de ubicaciones donde se pueden replicar datos. Cada zona tiene: un zonemaster, que asigna datos a spanservers; entre 100 y varios miles de spanservers, que sirven datos a clientes; proxies de ubicación, que ayudan a los clientes a encontrar los spanservers correctos.
- Universe master y placement driver: el primero es una consola de estado para depuración; el segundo mueve datos automáticamente entre zonas cada pocos minutos para cumplir restricciones de replicación o balancear carga.
- Spanserver: es el componente central. Cada spanserver maneja entre 100 y 1000 tablets. Un tablet almacena un mapeo: `(key:string,timestamp:int64)→string`. El estado del tablet se guarda en archivos tipo B-tree y un write-ahead log en Colossus.
- Además, cuenta con: replicación mediante maquinas de estados Paxos, control de concurrencia con locks, y una abstracción de directorios para colocar y mover datos globalmente.

¿Cómo multi versionamiento en Spnnaer?

- Cada versión tiene timestamp de commit: De modo que Spanner asigna a cada transacción un timestamp significativo. El timestamp refleja el orden de serialización y se usa como timestamp de commit. El mapeo interno es: `(key,timestamp)→string`.
- Versiones antiguas retenidas: Las versiones antiguas se pueden conservar según políticas configurables de garbage collection. Además, las aplicaciones pueden leer datos en timestamps antiguos.

- Un cliente puede hacer una snapshot read especificando un timestamp o una cota de antigüedad. La lectura se ejecuta sin locks en cualquier réplica que esté suficientemente actualizada.
- Cuenta con Snapshot transactions, que son transacciones predeclaradas de solo lectura. Se les asigna un timestamp elegido por el sistema y se ejecutan sin locks, sin bloquear escrituras entrantes.
- Se define como Safe Time al máximo timestamp hasta el cuál una réplica está actualizada
- Los timestamps permiten que si se hace una transacción previa a otra, el timestamp de la primera sea menor al de la segunda, manteniendo consistencia externa.
- Esto permite que hayan backups consistentes, ejecuciones MapReduce consistentes, cambios atómicos de esquema, lecturas no bloqueantes del pasado y transacciones snapshot sin locks.

¿En qué consiste el TRUETIME?

TrueTime es una API y una implementación de tiempo global creada por Google. Su característica central es que expone explícitamente la incertidumbre del reloj, en lugar de dar una hora exacta, de modo que se da un intervalo con un margen de error. Tiene 3 métodos principales:

- `TT.now()`: devuelve un intervalo `TTInterval`: [earliest, latest] que garantiza contener el tiempo absoluto en el momento de la invocación.
- `TT.after(t)`: devuelve true si el tiempo `t` ya pasó con certeza.
- `TT.before(t)`: devuelve true si el tiempo `t` todavía no ha llegado con certeza.

El intervalo tiene una incertidumbre acotada. Si ϵ es la mitad del ancho del intervalo, TrueTime garantiza que el tiempo absoluto está dentro de ese margen. Se mantiene entre 1 a 7 ms, con promedio de 4 ms.

Se utiliza en Spanner, en donde después de revibir solicitudes de commit se le asigna un timestamp `s` a estas, antes de exponer datos, el lider espera hasta que `TT.after(s_i)` sea verdadero. Como se menciona anteriormente se cuenta con consistencia extenerna pues cada commit tiene un `s` menor al del próximo commit, y en caso de que haya una gran incertidumbre el Spanner espera.